

Aqui está o seu guia completo! Desculpe a falta de sincronia anterior, acabei me empolgando em te acalmar e esqueci de colar o roteiro. Salve este texto, leia com atenção e imagine-se conversando com o professor. Se você entender a **estratégia** por trás do código, não precisará memorizar nenhuma linha de comando.

Guia de Defesa Oral: Projeto Senha (Java)

1. Visão Geral da Estrutura (O "Esqueleto" do Jogo)

O professor pode começar perguntando como o programa se organiza de forma geral. Você deve explicar que o sistema funciona em **dois grandes ciclos (loops)**:

- **Loop Principal (do-while):** Controla o jogo como um todo. Ele garante que, enquanto o usuário não escolher a **Opção 3 (Encerrar jogo)**, o programa continue ativo, permitindo jogar várias partidas seguidas.
- **Loop da Partida (while):** Controla uma partida individual. Ele roda enquanto a partidaAtiva for verdadeira. Essa partida só acaba se o jogador **descobrir a senha, esgotar as 8 tentativas** ou **desistir (Opção 2)**.
- **Restrições Técnicas:** Enfatize que, atendendo aos requisitos, você usou **apenas vetores unidimensionais** de inteiros (int[]), não usou String para lógica de comparação e centralizou tudo em funções/procedimentos (public static) dentro da classe principal.

2. Fluxo Principal da Partida (Passo a Passo)

Explique o que acontece na memória do computador quando o jogo inicia:

1. **Geração da Senha:** Assim que uma nova partida começa, o subprograma gerarSenha() cria um vetor de 6 posições preenchido com números aleatórios de 1 a 6.
2. **Exibição do Menu:** Dentro do loop da partida, o menu é impresso mostrando o status atual (número da partida, moedas acumuladas e tentativas restantes).
3. **Tratamento das Opções:**

- **Opção 1 (Tentativa):** O programa lê o palpite do usuário, processa os acertos, desconta uma tentativa e verifica se o jogo acabou (ganhou ou perdeu por tentativas).
- **Opção 2 (Desistir):** Exibe a senha, subtrai 80 moedas, encerra a partida atual e reinicia uma nova.
- **Opção 3 (Encerrar):** Desfaz a contagem da partida atual (já que ela não foi concluída) e quebra os laços de repetição para exibir o relatório final.

3. A Lógica Crítica: O Cálculo de V e Q (O que o professor vai focar)

Essa é a parte mais complexa e a que os professores mais testam. Eles querem saber como você evitou a **recontagem de números repetidos**. Explique assim:

Como funciona o cálculo de V (Posição Correta)

É um processo direto. Um laço for percorre o vetor da senha e o da tentativa ao mesmo tempo (mesmo índice i). Se `senha[i] == tentativa[i]`, a variável v é incrementada.

Como funciona o cálculo de Q (Posição Incorreta - Sem repetição)

Para calcular Q sem errar com números repetidos, a estratégia foi a **anulação de elementos**.

1. **Cópia de Segurança:** Primeiro, criamos cópias dos vetores da senha e da tentativa para não estragar os dados originais.
2. **Primeira Passada (Anular os "V"):** Rodamos um loop para identificar onde houve acerto exato (V). Nessas posições, "anulamos" os números, substituindo o valor na senha por 0 e na tentativa por -1. Isso garante que uma posição que já pontuou em V nunca seja contada de novo em Q.
3. **Segunda Passada (Contar os "Q"):** Usamos dois laços for aninhados (um dentro do outro). O de fora percorre a senha e o de dentro percorre a tentativa.
4. **A Queima de Cartucho:** Se encontrarmos um número igual (e que não seja os anulados 0 ou -1), incrementamos q. **Imediatamente**, anulamos essa posição na senha (virando 0) e na tentativa (virando -1) e usamos um break para parar de procurar naquele palpite. Isso impede que o mesmo número da senha pontue para dois números iguais da tentativa.

4. Resumo dos Subprogramas (Funções)

Se ele apontar para um método e perguntar "O que este faz?", responda de forma direta:

- **gerarSenha():** Retorna um vetor de 6 posições preenchido usando `Math.random()`.
- **lerTentativa():** Lê os 6 números digitados pelo jogador e os devolve em um vetor.
- **processarTentativa():** Junta a leitura do palpite, chama os calculadores de V e Q, imprime o resultado na tela e retorna o valor de V.
- **calcularV()** e **calcularQ():** Isolam as lógicas de contagem de acertos que explicamos acima.
- **verificarSenhaDescoberta():** Uma função booleana simples que retorna verdadeiro se `V == 6`.
- **verificarSePartidaTerminou():** Retorna verdadeiro se o jogador acertou a senha (`V == 6`) **OU** se as tentativas chegaram a zero.
- **calcularMoedas():** Recebe o número da tentativa atual e, usando uma estrutura de ifs, devolve a quantidade de moedas correspondente à tabela do enunciado.
- **exibirEstatisticas():** Recebe todas as variáveis globais acumuladoras e imprime o relatório final formatado quando o jogo fecha.



Dicas de Ouro para a Apresentação Oral

- **Use termos técnicos corretos:** Em vez de falar "o negócio que guarda os números", fale "**vetor**" ou "**arranjo**". Em vez de "pedaço de código separado", diga "**subprograma**", "**função**" (quando retorna valor) ou "**procedimento**" (quando é void).
- **Explique a Passagem de Parâmetros:** Se o professor perguntar por que você passa (senha, tentativa) para as funções, explique que *os vetores são locais e, para que uma função consiga analisar os dados da outra, esses dados precisam ser enviados como argumentos (parâmetros)*.
- **Seja confiante sobre as variáveis globais:** Mostre que as variáveis de estatísticas (como `partidasVencidas`, `moedasAcumuladas`) ficam declaradas no escopo do main porque elas precisam sobreviver ao longo de todas as partidas, enquanto as variáveis de dentro do while nascem e morrem a cada rodada. Com essa linha de raciocínio na cabeça, você domina a explicação de ponta a ponta.

Boa sorte na apresentação! Se quiser simular alguma pergunta que acha que ele faria, é só mandar.